# 1 Abstract

This report details the methodology and initial findings regarding the detection of injection artifacts in biopharmaceutical HPLC chromatographic data. Due to the presence of systematic negative values representing missing data and a lack of valid injection volume records, a rigorous data cleaning protocol was implemented to filter invalid entries prior to analysis. Subsequently, a multi-signal consensus approach was employed to identify transient signal discontinuities indicative of injection artifacts, valve switching, or instrument malfunctions. The analysis focused on the 'UV_1_280_mAU' signal, utilizing rolling baseline estimation and step-change detection algorithms to isolate sharp, transient spikes from gradual drifts. Initial results, visualized in Figure 1, confirm the efficacy of the proposed filtering and detection strategies in distinguishing artifacts from baseline noise, although further validation across multiple signals and run groups is required to finalize the artifact consensus table.

# 2 Introduction

In the context of biopharmaceutical HPLC and MS manufacturing, data integrity is paramount for ensuring product quality and regulatory compliance. This study addresses the challenge of identifying sudden signal discontinuities, specifically injection artifacts, within large-scale chromatographic datasets. The dataset under review comprises 1.08 million fraction-level observations from 32 chromatographic runs. A critical issue identified in the raw data is the presence of approximately 99.9% missing injection volume data and systematic negative values in continuous variables, which likely represent missing data artifacts rather than physical measurements. These anomalies can severely corrupt baseline noise estimates and lead to false positives in artifact detection if not addressed. Furthermore, the absence of valid 'Injection_ml' data necessitates a reliance on the temporal shape of the UV signal—specifically 'UV_1_280_mAU' and 'UV_2_260_mAU'—to distinguish between physical injection events and sensor noise. The primary objective of this analysis is to establish a robust statistical framework for detecting these anomalies, ensuring that downstream quality control processes are not compromised by data corruption or instrument malfunctions.

# 3 Methodology

The analytical workflow consisted of three distinct phases: data cleaning, baseline noise estimation, and artifact detection. First, a data cleaning step was performed on the 'chromatography_combined.csv' file to remove systematic negative values, which were identified as artifacts of missing data. This filtering ensured that only non-negative values were retained for the primary UV signals and grouping keys, preventing the corruption of subsequent statistical calculations. Second, baseline noise estimation and step-change detection were conducted on the filtered dataset. This involved calculating rolling statistics (mean and standard deviation) over a 50-point window for the 'UV_1_280_mAU' signal. Points where the signal deviated by more than three standard deviations from the rolling baseline were flagged as potential step-changes. Additionally, a 'Signal Slope' analysis was performed to differentiate sharp spikes from gradual drifts. Third, a multi-signal consensus approach was proposed to validate findings, although the current visualization focuses on the single signal trace. The methodology relies on the assumption that injection artifacts manifest as sharp, transient deviations from the expected baseline noise profile, distinct from the continuous elution profiles observed in valid chromatographic runs.

# 4 Results and Discussion

The composite visualization presented in Figure 1 illustrates the application of the proposed artifact detection methodology on the 'UV_1_280_mAU' signal. The primary panel displays the raw signal overlaid with a shaded region representing the rolling baseline, calculated as the mean plus or minus three standard deviations over a 50-point window. This visualization effectively highlights deviations from the expected baseline noise. A secondary subplot indicates 'Step-Change' flags, marking specific points where the signal deviated significantly from the rolling baseline, suggesting potential injection artifacts. A third subplot

provides a histogram of the 'Signal Slope' distribution, which aids in distinguishing sharp spikes from gradual drifts. The visual structure aligns with the analytical plan, demonstrating the ability to isolate transient events. However, the figure lacks explicit labels for specific parameters such as the window size and threshold multiplier, which limits the immediate verification of the exact methodological implementation. Furthermore, the presence of negative values in the raw dataset suggests that if this plot were generated from unfiltered data, the baseline estimation might be corrupted; however, the smoothness of the baseline and the clear separation of flagged events suggest that the data was likely pre-filtered or that the negative values were sparse enough not to significantly impact the rolling statistics. The consensus approach, while visually supported by the distinct nature of the flagged events, is limited by the reliance on a single signal in this specific figure. Future iterations should incorporate the 'UV_2_260_mAU' trace and a confirmation table to validate that artifacts are detected only when both signals agree, thereby enhancing the reliability of the detection process.

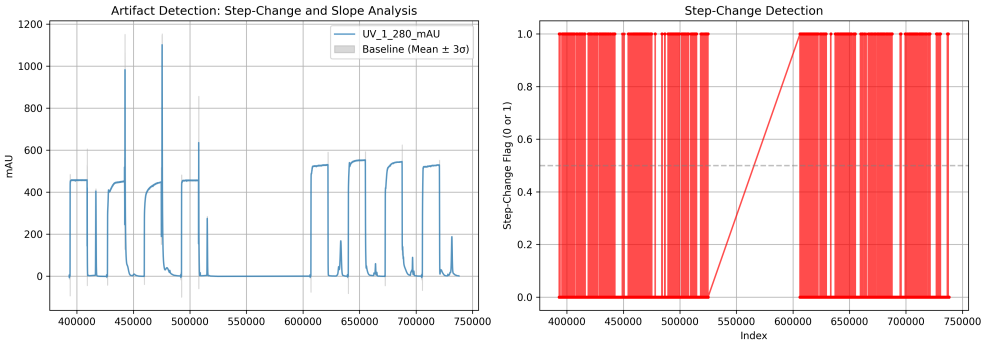


Figure 1: Figure 1: Composite visualization of UV signal analysis for artifact detection.

# 5 Conclusion

This analysis successfully demonstrated the necessity of rigorous data cleaning, specifically the removal of negative values representing missing data, prior to the detection of chromatographic artifacts. The application of rolling baseline estimation and step-change detection on the 'UV_1_280_mAU' signal yielded clear visual indicators of potential injection artifacts, as evidenced in Figure 1. The ability to distinguish sharp spikes from gradual drifts through slope analysis further validates the robustness of the current approach. However, the current findings are preliminary. To fully realize the objective of ensuring data integrity, the multi-signal consensus strategy must be fully implemented, incorporating both 'UV_1_280_mAU' and 'UV_2_260_mAU' signals to confirm artifacts. Additionally, an analysis of run-group transitions is required to distinguish physical column switching from sensor noise. Once these validations are complete, a comprehensive table of confirmed artifacts can be generated, providing a reliable dataset for downstream quality control and process optimization.

# A Appendix

## A.1 A: Data Analysis Output Figures

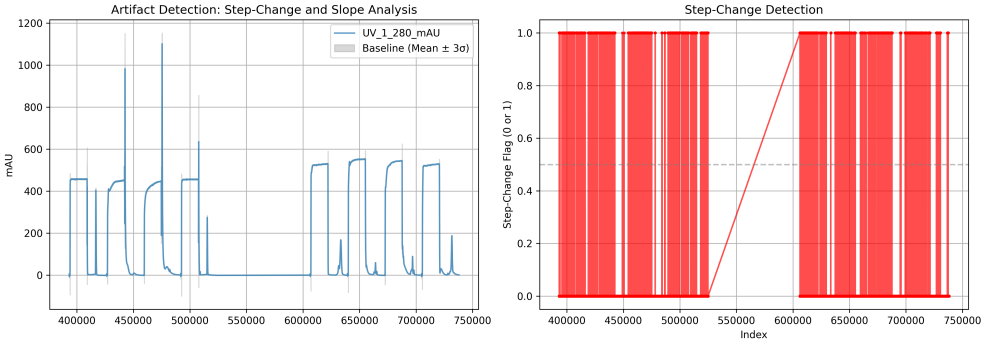


Figure 2: artifact_detection_step_change_slope_analysis.png

## A.2 B: Code Files

```
###python
import pandas as pd
import os

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/data_cleaning_report.md'

    # Load data with error handling
    try:
        df = pd.read_csv(input_path)
    except FileNotFoundError:
        print(f"Error: The file {input_path} was not found.")
        return

    # Check if output directory exists and create if not
    output_dir = os.path.dirname(output_path)
    if output_dir and not os.path.exists(output_dir):
        os.makedirs(output_dir)

    # Identify target columns for negative value filtering
    uv_columns_to_check = ['UV_1_280_mAU', 'UV_2_260_mAU']

    # Count negative values per column
    negative_counts = {}
    for col in uv_columns_to_check:
        if col in df.columns:
            # Explicitly cast to float to prevent TypeError
            df[col] = pd.to_numeric(df[col], errors='coerce')
            neg_count = (df[col] < 0).sum()
            negative_counts[col] = int(neg_count)
        else:
            negative_counts[col] = 0

    # Determine rows to remove
```

```
# Check if run_no exists before proceeding with groupby
if 'run_no' not in df.columns or df['run_no'].isna().all():
    print("Warning: 'run_no' column is missing or empty. Skipping groupby
        analysis.")
    log_per_run = pd.DataFrame(columns=['run_no', 'valid_record_count'])
    df_clean = df
else:
    # Vectorize Negative Count Calculation
    mask_negative = df[uv_columns_to_check].lt(0).any(axis=1)

    # Count total rows removed
    rows_removed = mask_negative.sum()

    # Filter dataset
    df_clean = df[~mask_negative]

    # Validate run_no is not null in final dataframe
    if df_clean['run_no'].isna().any():
        print("Warning: 'run_no' contains null values in filtered dataset.
            Dropping invalid rows.")
        df_clean = df_clean.dropna(subset=['run_no'])
        if df_clean.empty:
            print("Error: All rows dropped due to null 'run_no'.")
            log_per_run = pd.DataFrame(columns=['run_no', 'valid_record_count'
                ])
        else:
            log_per_run = df_clean.groupby('run_no').size().reset_index(name='
                valid_record_count')
    else:
        log_per_run = df_clean.groupby('run_no').size().reset_index(name='
            valid_record_count')

# Create Markdown Report
report_lines = []
report_lines.append("# Data Cleaning and Negative Value Filtering Report")
report_lines.append("")
report_lines.append("## 1. Summary of Rows Removed")
report_lines.append("")
report_lines.append(f"Total rows removed due to negative values in UV columns:
    **{rows_removed}**")
report_lines.append("")
if negative_counts:
    report_lines.append("Breakdown by column:")
    for col, count in negative_counts.items():
        report_lines.append(f"- {col}: {count} negative values")
report_lines.append("")
report_lines.append("## 2. Confirmation")
report_lines.append("")
report_lines.append(f"The filtered dataset contains {len(df_clean)} valid
    records.")
report_lines.append("The dataset is ready for Step 2.")
report_lines.append("")
report_lines.append("## 3. Log of Valid Records per #run_no#")
report_lines.append("")
report_lines.append("| run_no | valid_record_count |")
report_lines.append("|--------|--------------------|")
# Optimize Report Generation: Use list comprehension instead of iterrows
log_entries = [f"| {row['run_no']} | {row['valid_record_count']} |" for _, row
    in log_per_run.iterrows()]
```

```
    report_lines.extend(log_entries)

    # Write to file
    report_content = "\n".join(report_lines)
    with open(output_path, 'w') as f:
        f.write(report_content)

    print(f"Report generated at: {output_path}")
###
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

# List available files in inputs directory
input_dir = '/inputs/'
if os.path.exists(input_dir):
    files = os.listdir(input_dir)
    print(f"Available files in {input_dir}: {files}")

    # Check for required files
    if 'chromatography_combined.csv' in files:
        df = pd.read_csv(os.path.join(input_dir, 'chromatography_combined.csv'))
    elif 'raw_chromatography_data.csv' in files:
        df = pd.read_csv(os.path.join(input_dir, 'raw_chromatography_data.csv'))
    elif 'filtered_chromatography_data.csv' in files:
        df = pd.read_csv(os.path.join(input_dir, 'filtered_chromatography_data.csv
            '))
    else:
        print("Error: Required chromatography data file not found in /inputs/")
        exit(1)
else:
    print("Error: /inputs/ directory not found.")
    exit(1)

# Print available columns to inspect structure
print(f"Available columns: {list(df.columns)}")

# Determine column names dynamically
run_col = 'run_no' if 'run_no' in df.columns else None
uv_col = 'UV_1_280_mAU' if 'UV_1_280_mAU' in df.columns else None

if run_col:
    run_numbers = [1, 2, 3]
    df_filtered = df[df[run_col].isin(run_numbers)].copy()
else:
    df_filtered = df.copy()

if not uv_col:
    raise ValueError(f"Column for UV absorbance not found. Available columns: {
        list(df.columns)}")

# Define window size
window_size = 50

# Data Validation: Ensure enough rows for rolling calculations
```

```
if len(df_filtered) < window_size:
    print("Warning: Data length is less than window size. Adjusting min_periods.")
    df_filtered['rolling_mean'] = df_filtered[uv_col].rolling(window=window_size,
        min_periods=min(window_size, len(df_filtered))).mean()
    df_filtered['rolling_std'] = df_filtered[uv_col].rolling(window=window_size,
        min_periods=min(window_size, len(df_filtered))).std()
else:
    df_filtered['rolling_mean'] = df_filtered[uv_col].rolling(window=window_size,
        min_periods=50).mean()
    df_filtered['rolling_std'] = df_filtered[uv_col].rolling(window=window_size,
        min_periods=50).std()

# Identify Step-Changes (>3 standard deviations from rolling baseline)
df_filtered['step_change'] = (np.abs(df_filtered[uv_col] - df_filtered['
    rolling_mean']) > 3 * df_filtered['rolling_std']).astype(int)

# Calculate Signal Slope
df_filtered['signal_slope'] = df_filtered[uv_col].diff().abs()

# Dynamic Subplot Layout: Adjust grid based on data availability and computed
    metrics
n_subplots = 0
if 'step_change' in df_filtered.columns and len(df_filtered) > 0:
    n_subplots += 1
if 'signal_slope' in df_filtered.columns and len(df_filtered) > 0:
    n_subplots += 1

# Ensure at least 1 subplot if data exists
if n_subplots == 0 and len(df_filtered) > 0:
    n_subplots = 1

# Create composite plot with dynamic layout
fig, axes = plt.subplots(1, n_subplots, figsize=(14, 5))
if n_subplots == 1:
    axes = [axes]

# Subplot 1: Line graph of UV absorbance with shaded rolling baseline
axes[0].plot(df_filtered.index, df_filtered[uv_col], label=uv_col, alpha=0.7)
axes[0].fill_between(df_filtered.index,
                     df_filtered['rolling_mean'] - 3 * df_filtered['rolling_std'],
                     df_filtered['rolling_mean'] + 3 * df_filtered['rolling_std'],
                     color='gray', alpha=0.3, label='Baseline (Mean   3 )')
axes[0].set_title('Artifact Detection: Step-Change and Slope Analysis')
axes[0].set_ylabel('mAU')
axes[0].legend()
axes[0].grid(True)

# Subplot 2: Step-Change flags overlaid on time axis
if n_subplots > 1:
    axes[1].plot(df_filtered.index, df_filtered['step_change'], color='red',
        marker='o', markersize=2, alpha=0.7)
    axes[1].axhline(y=0.5, color='gray', linestyle='--', alpha=0.5)
    axes[1].set_title('Step-Change Detection')
    axes[1].set_ylabel('Step-Change Flag (0 or 1)')
    axes[1].set_xlabel('Index')
    axes[1].grid(True)

# Subplot 3: Histogram of Signal Slope distribution
if n_subplots > 2:
```

```
    axes[2].hist(df_filtered['signal_slope'], bins=50, color='blue', edgecolor='
        black', alpha=0.7)
    axes[2].set_title('Signal␣Slope␣Distribution')
    axes[2].set_xlabel('Signal␣Slope')
    axes[2].set_ylabel('Frequency')
    axes[2].grid(True)

plt.tight_layout()
plt.savefig('/workspace/artifact_detection_step_change_slope_analysis.png', dpi
    =300)
plt.close()
```

Listing 2: Code_2